URL Shortener — Database Schema

Project: URL Shortener
Technology: Python, FastAPI, SQLAlchemy, SQLite, Pydantic
Location: `/home/dominic/Documents/fastapiurl`
Report #: 4/20

Database Engine: SQLite

Table: `urls`

```
class URL(Base):
    __tablename__ = "urls"
    id: int (PK)
    original_url: str
    short_code: str (unique, indexed)
    created_at: datetime
    clicks: int (default 0)
```

Indexes: `short_code` (unique)

Database Schema

Entity Relationship Diagram

```
URL Shortener Entity Relationships
├── User ──→ URLs/Posts/Tasks (1:N)
└── (module-specific entities)
```

Tables

Table: User

Column	Type	Constraints	Description
id	Integer	PK, auto-increment	Primary key
email	String(255)	UNIQUE, NOT NULL	User email
username	String(100)	UNIQUE, NOT NULL	Display name
hashed_password	String(255)	NOT NULL	bcrypt hash
created_at	DateTime	NOT NULL, default=now	Account creation

Column	Type	Constraints	Description
is_active	Boolean	default=True	Account status

Table: Module-specific

Column	Type	Constraints	Description
id	Integer	PK	Primary key
(varies by module)	-	-	-

Table: Module-specific

Column	Type	Constraints	Description
id	Integer	PK	Primary key
(varies by module)	-	-	-

Relationships

- User has many resources (1:N)
- Resources belong to User (N:1)
- Module-specific entities linked via foreign keys

Indexes

Table	Index	Columns	Type	Purpose
users	ix_user_email	email	UNIQUE	Fast login lookup
urls	ix_url_short_code	short_code	UNIQUE	Fast redirect
tasks	ix_task_user	user_id	NON-UNIQUE	User task listing

Migration Strategy

- Currently using SQLite with auto-create via `Base.metadata.create_all()`
- Schema changes require manual migration script
- Recommended: Alembic for version-controlled migrations
- For production: migrate to PostgreSQL with Alembic

Performance Notes

- All foreign keys should be indexed
- Text search uses LIKE queries (consider FTS5 for SQLite)
- Large text fields (content, description) may benefit from compression